Inheritance

What is Inheritance?

- Inheritance means one class takes properties and methods from another class.
- **Parent / Base class** → gives features
- **Child / Derived class** → receives and can add more features
- **Example:**
  Parents → surname, habits
  Child → gets them + has own behavior

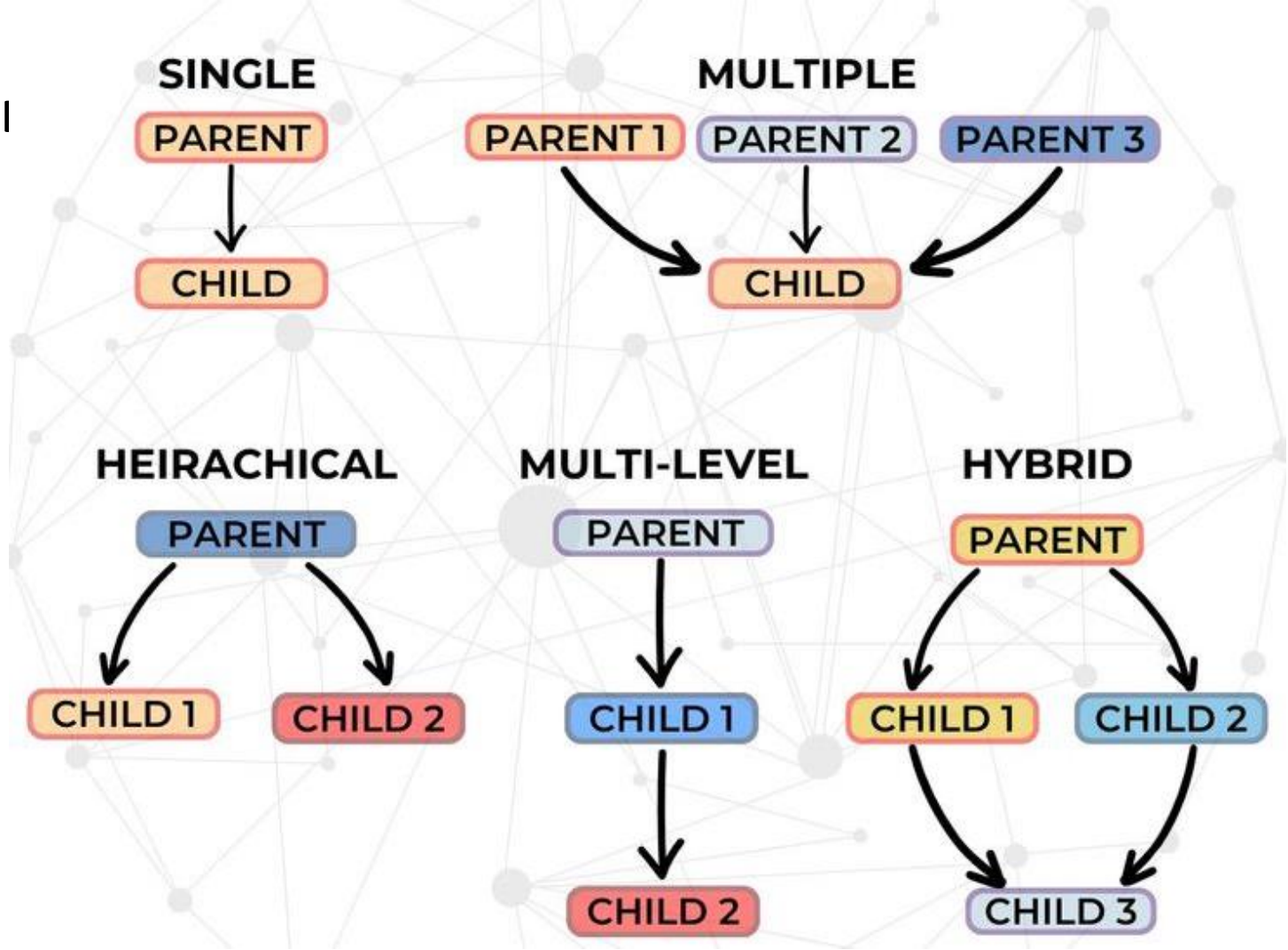
Why use inheritance?

- Code reusability
- Less duplication
- Easy to maintain
- Represents real-world relationships

Types of Inheritance in Python

Python supports **5 types of i**

- Single Inheritance
- Multiple Inheritance
- Multilevel Inheritance
- Hierarchical Inheritance
- Hybrid Inheritance



Single Inheritance

- One parent → one child

```python
# Parent class
class Animal:
    def eat(self):
        print("Animal is eating")

# Child class
class Dog(Animal):
    def bark(self):
        print("Dog is barking")

# Object creation
d = Dog()
d.eat()    # inherited method
d.bark()   # child method
```

Multiple Inheritance

- One child → multiple parents
- Python follows **MRO (left to right)**
- Father method is called first

```
Father: Driving
Child: Coding
```

```python
class Father:
    def skills(self):
        print("Father: Driving")

class Mother:
    def skills(self):
        print("Mother: Cooking")

class Child(Father, Mother):
    def own_skill(self):
        print("Child: Coding")

c = Child()
c.skills()      # Method Resolution Order (MRO)
c.own_skill()
```

Multilevel Inheritance

- Grandparent → Parent → Child
- Son gets everything from Father & Grandfather

```
Grandfather owns land
Father owns house
Son owns bike
```

```python
class Grandfather:
    def land(self):
        print("Grandfather owns land")

class Father(Grandfather):
    def house(self):
        print("Father owns house")

class Son(Father):
    def bike(self):
        print("Son owns bike")

s = Son()
s.land()
s.house()
s.bike()
```

Hierarchical Inheritance

- One parent → many children
- Car and Bike both inherit from Vehicle

```
Vehicle started
Car is driving
Vehicle started
Bike is riding
```

```python
class Vehicle:
    def start(self):
        print("Vehicle started")

class Car(Vehicle):
    def drive(self):
        print("Car is driving")

class Bike(Vehicle):
    def ride(self):
        print("Bike is riding")

c = Car()
b = Bike()

c.start()
c.drive()

b.start()
b.ride()
```

Hybrid Inheritance

- **Combination of two or more inheritance types**
- In this example, Uses **multiple + hierarchical**

```
Class A
Class B
Class C
Class D
```

```python
class A:
    def showA(self):
        print("Class A")

class B(A):
    def showB(self):
        print("Class B")

class C(A):
    def showC(self):
        print("Class C")

class D(B, C):
    def showD(self):
        print("Class D")

d = D()
d.showA()
d.showB()
d.showC()
d.showD()
```

super() keyword

- Used to call parent class constructor/method

```
class Parent:
    def __init__(self):
        print("Parent constructor")

class Child(Parent):
    def __init__(self):
        super().__init__()
        print("Child constructor")

c = Child()
```

Check inheritance

```
issubclass(Child, Parent)
isinstance(c, Child)
```